
16-709 - Robotics Foundations: Software Engineering

Assignment 4 – One Drawer, Two Controllers (MuJoCo, Gymnasium, SB3)

INSTRUCTIONS

- **Due:** [insert due date] at 11:59 PM EDT.
- **Weight:** This assignment makes up [insert %] of the overall course grade.
- **Scope:** One week. It covers the Lecture 6 material (MJCF composition, frames and sensing, numerical behaviour) and the Lecture 7 material (Gymnasium, reward design, Stable-Baselines3, evaluation protocol) on a single drawer-opening task.
- **Format:** Submit your code and output artifacts as a single archive on Gradescope (see **Submission**). **Instead of a written report, you will submit a short video** (see **Video**) explaining your work in your own words.
- **Starter code:** A code skeleton is provided in [Assignment-04/](#) alongside this handout; see its [README.txt](#) for file-by-file details, and run `setup_model.py` before anything else.
- **Policy:** See the course Canvas for homework policies and academic integrity.

Name	
Andrew ID	
Hours to complete?	☐ 0 - 2 hours ☐ 2 - 3 hours ☐ 3 - 4 hours ☐ 4 - 5 hours ☐ 5 - 6 hours ☐ 6 - 7 hours ☐ 7 - 8 hours ☐ > 8 hours

We know that tools like **ChatGPT** and **Claude** are readily available and, yes, parts of this assignment *are* technically solvable using them. But before you go outsourcing your brain, consider the following:

- You'll miss out on the core ideas this assignment is built around: why a model that compiles can still be physically wrong, why a Jacobian and a damped least-squares update behave the way they do near a singularity, and what a reward function actually specifies.
- A generated reward function or a generated environment will train. It will just train the wrong behaviour, and you will not be able to say why – which is exactly the skill this assignment is testing.
- You'll be unprepared for Q&A, or projects that require this knowledge – including your group project, which builds directly on these tools.
- This assignment asks for a short **video, in your own words**, explaining what you built and why. If you didn't actually build it, that conversation is a lot harder to have.

This isn't meant to scare you – just to remind you that learning takes effort, and short-circuiting that process won't help you in the long run. Use AI as a study aid if you must, but not as a substitute for understanding. We're excited for you to tackle the challenges ahead the right way.

Your 16-709 Course Staff

Objective

Build one MuJoCo scene properly, then solve the same task twice: once with a controller you derive, once with a policy you train. The robot is the Franka Emika Panda from MuJoCo Menagerie, the task is opening a drawer, and the success criterion is identical for both controllers – the drawer travels at least 0.24 m.

You will demonstrate proficiency with:

- **MJCF modelling and diagnosis:** composition through `<include>`, integration settings, collision filtering, mass ratios, and the difference between a model that runs and a model you can trust.
- **Frames and sensing:** a wrist camera placed by reasoning about conventions rather than by eye, and rendered RGB plus depth.
- **Classical control:** a site Jacobian, a damped least-squares resolved-rate update, and a task state machine.
- **Reinforcement learning:** a Gymnasium environment, a reward you design and defend, PPO from Stable-Baselines3, and a deterministic evaluation protocol.

The four sections form one pipeline: **fix** a model you can trust, **sense** the scene from the robot, **control** it analytically, then **learn** the same behaviour and compare the two honestly.

Scope

The scope is deliberately tight: no tree repair, no URDF-derived camera mount, no hyperparameter sweep. The habits are what matter – diagnose before you trust a model, validate before you spend compute, and report a protocol rather than a screenshot. Budget roughly the days below; the training runs are the only part you cannot compress.

Day	Work	Hours
1	Setup, diagnose and repair the scene, <code>inspect_scene.py</code> fully green	2.0
2	Wrist camera and RGB + depth capture	1.5
3	Resolved-rate controller (TODO 2.1 – 2.5)	2.5
4	Gymnasium environment (TODO 3.1 – 3.5), <code>check_drawer_env.py</code>	2.5
5	Three training runs, 150k steps each (mostly unattended)	1.0
6	Evaluation, video of the policy, comparison table	1.5
7	Record the 2-minute video, package the submission	1.0

Getting Set Up

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt

# the course MuJoCo tree already ships the Menagerie models; otherwise:
git clone https://github.com/google-deepmind/mujoco_menagerie

python setup_model.py --menagerie /path/to/mujoco_menagerie
```

`setup_model.py` copies the Menagerie Panda into `scene/models/` and writes `panda_task.xml`: the shipped keyframe removed, an `end_effector` site added, and `meshdir` rewritten. You must be able to explain all three edits.

The Starter Package

Every file you need is in `Assignment-04/`. Blue files are supplied and should run unedited; orange files hold the TODOs; you create `scene/drawer_scene.xml` yourself; grey entries appear when you run the scripts. `README.txt` repeats this as text.

Assignment-04/	
— Assignment-04.pdf	this handout
— README.txt	file-by-file guide and the day-by-day plan
— requirements.txt	mujoco, gymnasium, stable-baselines3, imageio
— setup_model.py	copies the Menagerie Panda into scene/models/
— scene/	
— environment.xml	floor, skybox, haze, lighting, task_view camera
— drawer_scene_bad.xml	loads cleanly and is wrong in five ways
— DEFECTS.txt	what to diagnose, fix, and report
— drawer_scene.xml	YOU create it: the repaired scene
— models/franka_emika_panda/	written by setup_model.py, not submitted
— section1/	model, diagnose, and sense
— inspect_scene.py	validator: 5 checks + stability + convergence
— capture_wrist.py	TODO 1.1 - 1.2 wrist RGB and depth
— output/	scene_report.txt, wrist_rgb.png, wrist_depth.*
— section2/	derive the controller
— drawer_controller.py	TODO 2.1 - 2.5 Jacobian, DLS, state machine
— output/	results.json, controller_plot.png
— section3/	learn the controller
— drawer_env.py	TODO 3.1 - 3.5 spaces, grasp, termination, reward
— check_drawer_env.py	check_env plus the random baseline
— train_ppo.py	TODO 3.6 evaluation callback
— evaluate_policy.py	TODO 3.7 deterministic protocol
— record_policy.py	offline video and interactive viewer
— runs/	monitor csv, tensorboard, best_model.zip
— output/	eval_results.json, eval_plot.png, policy.mp4
— section4/	compare the two controllers
— COMPARISON.txt	the table and five questions → comparison.json

■ supplied

■ the TODOs you complete

■ you create it

■ written when you run the scripts

Q1. [25 pts] Model, Diagnose, and Sense

A model that compiles is not a model that is sound. `scene/drawer_scene_bad.xml` loads without a single warning and is wrong in five ways.

Task 1.1 – Prepare the Menagerie model and compose the scene

- Run `setup_model.py`. It copies `mujoco_menagerie/franka_emika_panda` into `scene/models/`, writes `panda_task.xml`, and places the supplied `zed_cam_monocular.STL` into the model's `assets/` directory for Task 1.3. Keep the Menagerie directory, meshes, and `LICENSE` intact.
- Explain the three edits the script performs: why the shipped 9-value keyframe becomes invalid once the drawer joint exists, why an `end_effector` site is needed at all, and why `meshdir` must be rewritten when the main MJCF file lives in another directory.
- Report `nq`, `nv`, `nu`, the timestep, the integrator, and the world position of `handle.site` at the `home` keyframe.
- Keep the composition structure: robot, environment, and task objects in separate files combined with `<include>`. Do not flatten them into one monolithic XML.

Task 1.2 – Diagnose and repair five defects

The defects are marked with `DEFECT n` comments in `scene/drawer_scene_bad.xml`. The comment says *where* to look, not what is wrong. Save your repaired model as `scene/drawer_scene.xml`; Sections 2 and 3 load that file.

Area	What to report
<code><option></code> integration settings	symptom, why an explicit integrator has a timestep-dependent stability limit, your fix
<code>cabinet_shell</code> geom	the overlap with the drawer front, whether that pair should collide at all, and why it corrupts every later result
<code>drawer_slide</code> joint	which two attributes are missing, and what each one changes physically
drawer <code><inertial></code>	a body has <i>two</i> inertial properties; the mass is fine and its check passes. Compare the other one against the same property of the Panda links, and say what a ratio of that size does to the constraint solver. Enable the Inertia model element in the viewer's Rendering panel: MuJoCo then draws each body's equivalent inertia box, and this one is not subtle
<code>drawer_handle</code> geoms	the handle is a bracket built from two geoms. Report the size <i>and</i> the contact attributes of both, and say which one the gripper actually meets. Then compare that geom's size with the distance travelled per timestep at the approach speed, and with how many consecutive steps the pair can overlap at all

- For each defect give the observed symptom, the physical or numerical cause, and the value you chose with a reason.
- `python section1/inspect_scene.py` must print 5/5 static checks plus a green 10 s stability run and a green timestep-convergence test. Submit `section1/output/scene_report.txt`.
- Do not “fix” the scene by deleting the drawer, welding it shut, disabling gravity, or dropping the timestep below 0.0005. The repaired scene must be the same task.
- A passing check with no explanation earns no marks: the grading looks at the reasoning, not only at the diff.

Task 1.3 – Mount a wrist camera and photograph the drawer

Edit `scene/models/franka_emika_panda/panda_task.xml`. Use the values below verbatim, so that every submission renders from the same viewpoint and the grading is about your reasoning rather than about how patiently you nudged numbers.

- In `<asset>`, register the supplied mesh and a material for it. `setup_model.py` has already copied the STL into `assets/`, so `meshdir` resolves the bare file name:

```
<mesh name="zed_cam" file="zed_cam_monocular.stl"
      scale="0.001 0.001 0.001"/>
<material class="panda" name="zed_gray" rgba="0.20 0.20 0.24 1"/>
```

State what unit the STL is authored in and why the scale factor is exactly 0.001.

- Inside `<body name="hand">`, add a body named `cam_body` that carries *both* the camera and its physical shell, so the two can never drift apart. You write this block yourself. It needs four things:
 1. the body itself, mounted at `pos="0.048 0 0.01"` in the hand frame – use this value so every submission renders the same view;
 2. an `<inertial>` element giving the housing `mass="0.16"` (a real ZED X One) and a small diagonal inertia, around $1e-4$ per axis. Say what MuJoCo would do about the shell’s mass if you left this element out, and why that matters on an arm whose actuator gains were tuned for the shipped inertias;
 3. a `<geom type="mesh">` using the `zed_cam` mesh and the `zed_gray` material. It needs `euler="1.5708 0 0"` to stand the housing upright, and it belongs to `class="visual"` – say why a camera housing should not be a collision geom here;
 4. a `<camera name="wrist_cam">` offset `0.03` along the body’s own $+Z$, so the optical centre sits at the front face of the housing rather than inside it. Its orientation is the next part.

Aligning the camera axes

The one attribute not given to you is the camera’s orientation, because it is the part with the reasoning in it. Do not rotate the camera until the picture looks right – derive it from two conventions:

- a MuJoCo camera looks along its own $-Z$ axis, with $+Y$ up in the image;
- the Panda gripper approaches along the hand frame’s $+Z$.

So a wrist camera that sees what the gripper is about to grasp must have its own $-Z$ pointing along the hand’s $+Z$. Fix the roll as well, so that everyone’s images match: the image’s “up” direction must be the hand’s $+X$.

The `<camera>` element takes no angle. It takes

$$\text{xyaxes}="x_1\ x_2\ x_3\ y_1\ y_2\ y_3\ "$$

which are the camera’s own X and Y axes expressed in the parent frame, and MuJoCo completes the frame with $z = x \times y$. That is the tag you use to align a camera exactly, and it is why no trial and error is needed.

- Work out the `xyaxes` that satisfies both constraints, and show the derivation: write x and y as vectors in the hand frame, compute $z = x \times y$, and show that $-z$ is the hand’s $+Z$. Two constraints fix the frame completely, so there is exactly one right answer.

- **Demonstrate it, do not assert it.** Open the scene in the viewer, switch the view to `wrist_cam` (the camera selector is in the viewer's left panel; the [and] keys cycle cameras), and capture a screenshot showing both the arm and the view down the camera. Submit it as `wrist_view.png`. A camera pointing backwards produces a perfectly valid render of the wrong thing, and this screenshot is what proves you did not ship one.
- Complete TODO 1.1 and TODO 1.2 in `section1/capture_wrist.py` and save `wrist_rgb.png`, `wrist_depth.npy`, `wrist_depth.png`, and `wrist_pose.txt` at 640×480 .
- Show that the camera is rigidly attached: change the arm configuration, re-render, and report both camera world poses.

Matching the real camera

A MuJoCo `<camera>` defaults to `fovy="45"`, a vertical field of view in degrees. Real cameras are specified by a focal length and a sensor size instead, and MuJoCo accepts those directly through the `focal`, `sensorsize` and `resolution` attributes.

1. The mesh you mounted is a ZED X One 4K with a 3 mm lens. Look up its sensor size, and give the `fovy` that would match that real camera. Is the 45° default wider or narrower than the real thing?
2. Re-specify the camera with `focal`, `sensorsize` and `resolution` instead of `fovy`, reload, and report the `fovy` MuJoCo derives. In one sentence: why is this the form to use when simulated images are meant to train something that will later run on the real camera?

Verify

```
python -m mujoco.viewer --mjcf=scene/drawer_scene_bad.xml
python section1/inspect_scene.py # expect 5/5 plus both dynamic checks
export MUJOCO_GL=egl # osmesa on a CPU-only machine
python section1/capture_wrist.py
```

Common failure. A camera that is visible in the viewer is not a camera you can render from. A named `<camera>` plus explicit renderer selection is still required, and passing the wrong name silently returns the third-person view.

Deliverables

- `scene/drawer_scene.xml` and the modified `panda_task.xml`
- `section1/output/scene_report.txt`
- `wrist_rgb.png`, `wrist_depth.npy`, `wrist_depth.png`, `wrist_pose.txt`
- `wrist_view.png` – the viewer screenshot showing the camera pointing along the approach direction
- Completed `section1/capture_wrist.py`

Q2. [30 pts] Derive the Controller

The scene, viewer loop, keyframe reset, task phases, metrics, plotting, and JSON output are supplied in `section2/drawer_controller.py`. Complete the five TODOs that contain the control ideas. The equality weld after reaching the handle keeps the focus on feedback control rather than grasp-contact tuning.

Background

Before the TODOs, three ideas in plain terms. If you have seen them before, skim; if you have not, this is enough to do the assignment, and the references at the end point to the full treatment when you want it.

A feedback controller measures where the system is, compares that with where it should be, and turns the difference into a command. Here the measurement is the tool position x (m), the target is x_{des} (m), and the command is a joint position sent to the actuators. Note what is *not* here: no model of the arm's mass or gravity appears anywhere in the loop. The controller only ever sees an error and shrinks it, which is both why it is simple to write and why it can be fooled.

The Jacobian J is the matrix that converts joint rates into tool velocity,

$$v = J \dot{q},$$

where v is 3×1 (m/s), $\dot{q}$ is 7×1 (rad/s), and J is 3×7 (m/rad). It is the local linearisation of the forward kinematics: at the current pose, driving joint i at 1 rad/s moves the tool along column i of J . It is a property of the configuration, not of the robot alone, so it is recomputed every step.

Damped least squares answers the reverse question – which joint rates produce the tool velocity I want. That inverse is awkward twice over: J is not square (7 joints, 3 constraints, so infinitely many answers), and it loses rank in some poses (so the exact answer can demand infinite joint speed). One expression handles both:

$$\dot{q} = J^\top (JJ^\top + \lambda^2 I)^{-1} v,$$

with λ in m/rad. Away from a singularity the $\lambda^2 I$ term is negligible and this is the minimum-norm solution. Near one it dominates, and the controller gives up a little tracking accuracy in exchange for a bounded $\dot{q}$ instead of an unbounded one.

Further reading (optional). Siciliano & Khatib, *Springer Handbook of Robotics*, ch. 2 (kinematics) and ch. 8 (motion control); Lynch & Park, *Modern Robotics*, ch. 5 (velocity kinematics and the Jacobian) and §6.3 (numerical inverse kinematics) – the full text is free at <http://modernrobotics.org>; Buss, “Introduction to Inverse Kinematics with Jacobian Transpose, Pseudoinverse and Damped Least Squares Methods” (2009), a short note that covers exactly the expression above; Nakamura & Hanafusa (1986), the original singularity-robust inverse.

Task 2.1 – Resolved-rate motion

- **TODO 2.1:** the translational Jacobian of the supplied `end_effector` site, shape $3 \times \text{nv}$. The function you want is `mujoco.mj_jacSite`; no skeleton is given, so read its signature in the MuJoCo API reference and work out what it expects, what it writes into rather than returns, and which part of the result you need. Say in your video what the other part would have been for.
- **TODO 2.2:** the Cartesian position error $e = x_{\text{des}} - x$ the controller drives to zero.
- **TODO 2.3:** the damped least-squares update

$$\dot{q} = J^\top (JJ^\top + \lambda^2 I)^{-1} K e,$$

using `numpy.linalg.solve`. Do not form an explicit inverse, and explain what λ does near a singularity.

The supplied code then integrates your $\dot{q}$ into a persistent joint reference, `state.q_ref`, rather than re-deriving the command from the measured `qpos` each step. Read the comment above that line and be ready to say why the difference matters when the command is consumed by a position servo.

Task 2.2 – Task state machine

- **TODO 2.4:** choose the approach-to-grasp distance threshold and implement the transition. Say what a threshold that is too tight or too loose does.
- **TODO 2.5:** the success test at 0.24 m of drawer travel, completed within 12 s with a finite state, no joint-limit violation, and no sustained actuator saturation.

Task 2.3 – Evaluate the controller

- Submit `section2/output/results.json` and `section2/output/controller_plot.png`.
- Report completion time, maximum position error, maximum joint velocity, and maximum absolute control.
- Describe one gain choice that failed and classify the resulting behaviour as underdamped, overdamped, saturated, or unstable.

Verify

```
python section2/drawer_controller.py # with the interactive viewer
python section2/drawer_controller.py --headless # faster, no window
# expect: "Drawer opened at t=..." plus results.json and controller_plot.png
```

Deliverables

- Completed `section2/drawer_controller.py`
- `section2/output/results.json`, `section2/output/controller_plot.png`

Q3. [35 pts] Learn the Controller

Same scene, same success criterion, different source of the control law. MuJoCo computes the transition; Gymnasium defines the task; PPO writes the policy.

Task 3.1 – Define the learning problem

Complete TODO 3.1 to TODO 3.4 in `section3/drawer_env.py`. The observation layout is fixed so the supplied checks agree with your code.

Index	Contents	Size
0 – 6	arm joint positions, <code>joint1..joint7</code>	7
7 – 13	arm joint velocities, <code>joint1..joint7</code>	7
14 – 16	end-effector position, site <code>end_effector</code>	3
17 – 19	handle position minus end-effector position	3
20	drawer opening, <code>qpos[drawer_slide]</code>	1
21	drawer opening rate, <code>qvel[drawer_slide]</code>	1
22	grasped flag, 0 or 1	1

- **TODO 3.1:** build that observation as `float32`, and justify why the relative handle vector is included even though it is redundant.
- **TODO 3.2:** map the normalised action to the seven position actuators as a bounded increment of the current target, clipped to `actuator_ctrlrange`. No absolute joint commands, no writing to the gripper actuator.
- **TODO 3.3:** activate the weld the first time the tool is within `grasp_threshold` of the handle – the same decision you made in TODO 2.4, now inside the environment.
- **TODO 3.4:** terminate on success, and terminate on a non-finite state. “State” means MuJoCo’s own state vector, `data.qpos` and `data.qvel`, and it means *all* of it – not just the seven arm joints. The drawer slide and the two finger joints are integrated in the same step, so a divergence in any of them has already corrupted the transition you are about to return; `np.isfinite` over both arrays is the whole test. This is a check of numerical validity, not of bad behaviour: an arm moving fast, or a drawer slammed into its limit, is a poor episode but a perfectly finite one and must *not* terminate here. Say in your video what a policy would learn if you terminated on “large” instead of on “not finite”. The six-second budget is neither: it is *truncation* and belongs to the `TimeLimit` wrapper.

Task 3.2 – Design the reward and validate before training

Complete TODO 3.5. The supplied `RewardConfig` is a starting point, not a solution; every weight you change needs one sentence explaining the behaviour it fixes.

Term	Purpose	Starting weight
<code>reach</code>	shaping toward the handle before the grasp	1.0 per metre
<code>open</code>	drawer displacement, the actual objective	10.0 per metre
<code>grasp</code>	one-off bonus when the weld engages	5.0
<code>success</code>	one-off bonus at 0.24 m	50.0
<code>action_cost</code> / <code>action_rate_cost</code>	penalties on action magnitude and on chattering	0.01 / 0.01

- `cfg.sparse` must return the success term alone. That ablation is required in Task 3.3.

- Run `python section3/check_drawer_env.py`; SB3 `check_env` must pass and you must submit the output as `section3/output/env_check.txt`.
- Report the random-policy baseline (mean return, mean length, mean maximum opening, success rate). That is the number learning has to beat.
- Explain reward hacking with one concrete example from this task, such as hovering at the handle to farm the `reach` term without ever pulling.

Task 3.3 – Train and evaluate

- Complete TODO 3.6 (evaluation callback) and TODO 3.7 (deterministic evaluation episode).
- Train two seeds at 150k steps plus one sparse-reward run at the same budget. Training is headless; rendering during training is a defect.
- Predict what the sparse run will do before you launch it, then report what happened.
- Evaluate the better seed over 10 deterministic episodes and report success rate, mean and standard deviation of return, mean time to open, and the largest absolute action. Evaluate once with randomisation on and once off.
- Record `section3/output/policy.mp4` with the offscreen renderer, and describe one failure you observed – a bad seed, or an early checkpoint that reaches but never pulls.
- Submit the Monitor csv or TensorBoard logs for all three runs, plus wall-clock times.

Verify

```
python section3/check_drawer_env.py # check_env plus the baseline
python section3/train_ppo.py --seed 0
python section3/train_ppo.py --seed 1
python section3/train_ppo.py --seed 0 --sparse # the required ablation
python section3/evaluate_policy.py --model section3/runs/ppo_seed0/best_model.zip
python section3/record_policy.py --model section3/runs/ppo_seed0/best_model.zip
```

Common failure. An environment that trains for an hour and learns nothing is usually not an algorithm problem. It is an observation missing the quantity the reward depends on, an action scale that cannot reach the handle within the episode, or a termination that never fires. `check_drawer_env.py` catches all three in twenty seconds.

Deliverables

- Completed `section3/drawer_env.py`, `train_ppo.py`, `evaluate_policy.py`
- `output/env_check.txt`, `output/eval_results.json`, `output/eval_plot.png`, `output/policy.mp4`
- Logs for all three runs and the `best_model.zip` you evaluated

Q4. [10 pts] Compare the Two Controllers

You now have two controllers for one task, built from the same scene and judged by the same criterion. Section 2 numbers come from `section2/output/results.json`, Section 3 numbers from `section3/output/eval_results.json`.

Requirements

- Fill the table in `section4/COMPARISON.txt` and save it as `section4/comparison.json`: success rate, time to open, maximum control, maximum joint velocity, environment samples consumed, wall-clock compute, your engineering time, and success with cabinet jitter enabled.
- Run the classical controller once with the randomised cabinet to fill the last row, or state clearly that you did not.
- Answer the five questions in that file:
 1. **Sample cost.** The controller needed zero environment samples; the policy needed 150,000. What did you get in exchange – and did you actually get it on this task?
 2. **Where the knowledge lives.** In Section 2 it is in the Jacobian, the damped least-squares update, and the state machine you wrote. In Section 3 it is in the reward and the network weights. Which is easier to inspect when the robot does something unexpected?
 3. **The model matters to both.** Name one Section 1 defect that would have broken the classical controller, and one that would have quietly corrupted the RL result without making training crash.
 4. **Failure modes.** One failure specific to the learned policy that cannot happen to the resolved-rate controller, and one specific to the controller.
 5. **Your recommendation.** For this drawer, in a lab, next week: which one would you ship? What would have to change about the task for you to switch?

A defended “the classical controller wins here” earns full marks. An undefended “RL is better” does not.

Deliverables

- `section4/comparison.json`

Bonus (+10)

This replaces an assumption the required task makes for you. Show it in your video.

- **Pixel observations.** Feed the wrist camera into the policy through an SB3 `BaseFeaturesExtractor` instead of the state vector, and report the sample-efficiency cost against the state-based baseline. Say what the policy can no longer see, and what it can now see that the state vector never carried.

Video (replaces the written report)

Record **2 minutes or less**, in your own words – no script, no AI-written text. We are listening for understanding, not polish. Screen-recording your terminal, the MuJoCo viewer, or your plots while you talk is encouraged.

Cover the following:

1. What you built.
2. How the pieces connect: Menagerie model → repaired scene → wrist camera → resolved-rate controller → Gymnasium environment → PPO → deterministic evaluation.
3. One problem you hit and how you fixed it.
4. One thing you learned.

Follow the course policy on permitted assistance.

Submission Instructions

Submit a single compressed archive (`.tar.gz` or `.zip`) on Gradescope containing:

1. The model you repaired:
 - `scene/drawer_scene.xml`
 - the modified `panda_task.xml` (with your `wrist_cam`)
2. Section 1 outputs:
 - `section1/output/scene_report.txt`
 - `wrist_rgb.png`, `wrist_depth.npy`, `wrist_depth.png`, `wrist_pose.txt`
 - completed `section1/capture_wrist.py`
3. Section 2:
 - completed `section2/drawer_controller.py`
 - `output/results.json` and `output/controller_plot.png`
4. Section 3:
 - completed `section3/drawer_env.py`, `train_ppo.py`, `evaluate_policy.py`
 - `output/env_check.txt`, `output/eval_results.json`, `output/eval_plot.png`, `output/policy.mp4`
 - the Monitor or TensorBoard logs of all three runs, and the `best_model.zip` you evaluated
5. `section4/comparison.json`.
6. `versions.txt` from `pip freeze`.
7. Your video file, or a link to it in `video.txt`.

Do not submit `.venv/`, `scene/models/` (the copied Menagerie meshes), or intermediate checkpoints.

Grading Rubric

Criterion	Points
Section 1: Menagerie setup and scene composition explained, not just executed.	5
Section 1: five defects diagnosed with symptom, cause, and justified fix.	10
Section 1: <code>inspect_scene.py</code> fully green, report submitted.	3
Section 1: wrist camera derived correctly; RGB/depth captured and pose reported.	7
Section 2: site Jacobian, Cartesian error, and damped resolved-rate update correct.	12
Section 2: grasp transition, smooth pull, limits, and success test correct.	8
Section 2: metrics, plot, and one diagnosed gain failure.	5
Section 2: drawer opens at least 0.24 m within 12 s; simulation stays finite.	5
Section 3: observation, action mapping, grasp, and termination correct and justified.	10
Section 3: reward terms implemented including a working sparse variant; baseline reported.	8
Section 3: two seeds plus the sparse ablation, trained headlessly and logged.	9
Section 3: deterministic evaluation over 10 episodes, video, and a diagnosed failure.	8
Section 4: comparison table complete and the five questions defended.	6
Video: 2 minutes or less, unscripted, covers all four required points.	4
Total	100
Bonus: pixel observations – shown in the video.	+10